

are the same as the host system. To add your own network stack, you need to connect to the existing network bridge.

```
#systemd-nspawn -D my_deb --network-bridge=br0
```

This will start the container with the network namespace with a pair of *veth* devices. You can even boot the instance by the *-b* option, as follows:

```
#systemd-nspawn -bD my_deb
```

 **Note:** While booting the container, you will be required to enter the password of the root user; so first run *#passwd* inside to set the root password.

The whole *nspawn* project is relatively young; hence there is still a lot that needs to be developed.

Docker

Docker is the smartest and most prominent container in Linux to run an applications environment. Over the past few years, it has grabbed the most attention. Docker containers use most of the namespaces and cgroups present in *systemd-nspawn*, in which any service can be deployed with just a few tweaks. It can be used as a sandboxing tool to run applications securely or to deploy some software service inside it.

To get your first Docker container running, you need to first start the Docker daemon, and then download the base image from the Docker online repository, as follows:

```
#service docker start
#docker pull kalilinux/kali-linux-docker
```

 **Note:** You can also download other Docker images from the Docker Hub (<https://hub.docker.com/>).

It will download the base Kali Linux image. You can see all the available images on your system by using the following code:

```
#docker images
REPOSITORY TAG IMAGE ID CREATED VIRTUAL SIZE
kalilinux/kali-linux-docker
latest 63ae5ac8df0f 1 minute ago 325 MB
centos centos6 b9aeae5e17 9 months ago 202.6 MB
hello-world latest 91c95931e552 9 months ago 910 B
```

To run a program inside your container, use the command given below:

```
#docker run -i -t kalilinux/kali-linux-docker ls
bin dev home lib64 mnt proc run selinux sys usr
```

```
boot etc lib media opt root sbin srv tmp var
```

This will start (run) your container, execute the command and then close the container. To get an interactive shell inside the container, use the command given below:

```
#docker run -t -i kalilinux/kali-linux-docker /bin/bash
root@24a70cb3095a:/#
```

This will get you inside the container where you can do your work, isolated from your host machine. *24a70cb3095a* is your container's ID. You can check all the running containers by using the following command:

```
#docker ps
CONTAINER ID IMAGE STATUS PORTS COMMAND
CREATED NAMES
24a70cb3095a kalilinux/kali-linux-docker "/bin/ bash" About a minute ago Up About a minute
angry_cori
```

While installing the Docker image, Docker automatically creates a *veth* for Docker, which makes the Docker image connect to the main system. You can check this by using *#ifconfig* and ping your main system. At any instance, you can save your Docker state as a new container by using the code given below:

```
#docker commit 24a70cb3095a new_image
#docker images
REPOSITORY TAG IMAGE ID CREATED VIRTUAL SIZE
new_image latest a87c73abca9d 6 seconds ago 325 MB
kalilinux/kali-linux-docker
latest 63ae5ac8df0f 1 hours ago 325 MB
centos centos6 b9aeae5e17 9 months ago 202.6 MB
hello-world latest 91c95931e552 9 months ago 910 B
```

You can remove that image by using *#docker rmi new_image*. To stop a container, use *docker stop* and after that remove the files created on the host node by that container.

```
#docker stop 24a70cb3095a
#docker rm 24a70cb3095a
```

For running applications on a Docker instance, you may require to attach it to the host system in some way. So, to mount the external storage to the Docker image, you can use the *-v* flag, as follows:

```
#docker run -it -v /temp:/home/ kalilinux/kali-linux-docker /bin/bash
```

This will mount */temp/* from the main system to the */home/*